



Definitive: Solana Trading Security Review

Cantina Managed review by:
0xHuy0512, Security Researcher
Arno, Associate Security Researcher

June 18, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Flash pre-swap and post-swap are not bound to the same group	4
3.1.2	Performer revocation can silently fail	4
3.2	Low Risk	4
3.2.1	Critical order terms are hidden from the user signature prompt	4
3.2.2	Under-scoped flash order signatures allow swaps without user protection	5
3.2.3	WSOL output fee recipient branch is unreachable	5
3.2.4	Out-of-range Flash output fee rates can wrap on cast instead of failing closed	6
3.2.5	Unsigned group selection can route Flash integrator fees into a performer-owned group	6
3.2.6	Flash swap Ed25519 verification excludes multisig token owners	7
3.3	Informational	7
3.3.1	FlashOrders initialization can be blocked by pre-funding the PDA	7
3.3.2	Redundant performer validation increases flash swap complexity	8
3.3.3	Deprecated library method is used during FlashOrders migration	8
3.3.4	Flash swap cache is left dirty after post-swap	8
3.3.5	Performer role is over-broad	9
3.3.6	Flash events attribute swaps to the performer and report gross output	9
3.3.7	Flash swap user signature does not bound output amount	9

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Definitive Finance delivers a CeFi-like experience on DeFi rails via a fully non-custodial platform & API that is live across Solana, Base, HyperEVM and all other major EVM chains.

From May 18th to May 25th the Cantina team conducted a review of [solana-trading](#) on commit hash [ee34a3e8](#). The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	3	0
Low Risk	5	3	2
Gas Optimizations	0	0	0
Informational	7	4	3
Total	15	10	5

2.1 Scope

The security review had the following components in scope for [solana-trading](#) on commit hash [ee34a3e8](#):

```
programs/trading/src
├── lib.rs
├── handlers
│   ├── handle_close_flash_nonce.rs
│   ├── handle_flashpostswap.rs
│   ├── handle_flashpostswapsim.rs
│   ├── handle_flashpreswap.rs
│   ├── handle_flashpreswapsim.rs
│   ├── handle_init_flash_nonce_tracker.rs
│   └── handle_migrate_flash_orders.rs
├── state
│   ├── flash_nonce.rs
│   └── swap.rs
└── utils
    ├── consts.rs
    ├── ed25519.rs
    └── swap.rs
```

3 Findings

3.1 Medium Risk

3.1.1 Flash pre-swap and post-swap are not bound to the same group

Severity: Medium Risk

Context: [handle_flashpostswap.rs#L43](#)

Description: `flash_pre_swap` and `flash_post_swap` are not forced to use the same `group` .

During `flash_pre_swap` , the program saves temporary settlement data into the supplied group's `active_swap_data` . Later, during `flash_post_swap` , the program reads settlement data from whatever group is supplied to the post-swap instruction.

The problem is that neither the signed Flash order nor the instruction-ordering checks prove that both instructions use the same group. So a valid performer can run a victim's pre-swap using one group, then run post-swap using another group that already has stale settlement data.

This can cause settlement to use the stale group's stored destination instead of the destination from the victim's actual pre-swap.

Recommendation: Bind `flash_pre_swap` and `flash_post_swap` to the same group and session.

Definitive Finance: Fixed in [PR 102](#).

Cantina Managed: [PR 102](#) fixes this by enforcing that `flash_pre_swap` and `flash_post_swap` use the same group account. A proof-of-concept confirmed that a performer could route a victim's pre-swap output through a different group's stale settlement state, causing the victim's signed destination to receive nothing while an attacker-controlled destination received the full output.

3.1.2 Performer revocation can silently fail

Severity: Medium Risk

Context: [handle_flashpreswap.rs#L216](#)

Description: `MetaConfig` can store the same performer key more than once in `secondary_performers` . It can also store a key as both the primary performer and a secondary performer.

When `RemoveSecondaryPerformer` is called, it only removes the first matching secondary performer entry. If the same key exists again in another slot, that key is still considered a valid performer by `valid_performer()` .

This means an operator may think a performer key was revoked, but the key can remain authorized through a duplicate or shadow entry.

Recommendation: Ensure performer entries are unique.

Definitive Finance: Fixed in [PR 99](#).

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Critical order terms are hidden from the user signature prompt

Severity: Low Risk

Context: [handle_flashpreswap.rs#L76-L88](#)

Description: In `handlers/handle_flashpreswap.rs`, `flash_pre_swap()` builds `expected_message` as `DFS|m=<input_mint>|t=<order_total>|h=<order_hash>`. The `order_hash` commits to `output_token_account`, `nonce`, and `deadline`, but those values are not visible in the message the end user signs. The performer constructs and submits the actual transaction, while the user only signs the Ed25519 message.

This means the user cannot inspect the real `output_token_account` or `deadline` through the wallet approval surface. The hash prevents the performer from changing those fields after the signature is created, but it does not prove the user understood which destination and expiry were authorized. This will lead to users signing opaque hashes produced by the performer and authorizing a swap whose output destination or validity window differs from the user's expectation.

Recommendation: Make critical order terms visible in the signed message. Include `output_token_account`, `deadline`, and preferably `nonce` as plain fields in `expected_message`, not only inside `order_hash`.

Definitive Finance: Acknowledged.

Cantina Managed: Definitive Finance acknowledged the trade-off between human-readable signature fields and transaction size constraints. The team noted that the majority of Flash API users are agentic or use embedded wallets, where human-readable fields are less critical. They committed to adding validation that `output_token_account` must be owned by the signing wallet, addressed in [PR 90](#). Because the added ownership check is unrelated to expanding the visible signature fields, this finding remains acknowledged.

3.2.2 Under-scoped flash order signatures allow swaps without user protection

Severity: Medium Risk

Context: `handle_flashpreswap.rs#L76-L88`

Description: In `handlers/handle_flashpreswap.rs`, `flash_pre_swap()` verifies that the swapper signed `expected_message`, but the message only covers `input_mint`, `order_total`, and a hash containing `output_token_account`, `nonce`, and `deadline`.

Economically important terms are supplied outside the signed message:

- The same owner can approve multiple token accounts for the same mint, and because the signature does not bind `swapper_token_account`, the performer can choose an unexpected approved input account for the order.
- Without `program_id`, the signature is also weaker against cross-program signature replay.

Recommendation: Expand the signed order message to cover all user security assumptions by including `swapper_token_account`, and `program_id`.

Definitive Finance: Fixed in [PR 89](#).

Cantina Managed: [PR 89](#) fixes this by adding `program_id` and `input_token_account` to the `order_hash`, binding those fields to the user's signature. The `min_output_amount` is intentionally excluded due to the trusted performer assumption and the multi-fill order model; a separate informational finding covers that design decision.

3.2.3 WSOL output fee recipient branch is unreachable

Severity: Low Risk

Context: `handle_flashpostswap.rs#L32-L35`

Description: In `handlers/handle_flashpostswap.rs`, `process()` first rejects all remaining accounts with `ctx.remaining_accounts.len() == 0`. Later, the WSOL output-fee branch checks `ctx.remaining_accounts.len() > 0` before allowing the performer ATA to receive the protocol fee. Because the earlier check guarantees there are no remaining accounts, this branch can never execute. This will lead to an ineffective special case for WSOL output fees, causing fees to always route through the default `output_fee_token_account`.

Recommendation: Either remove the unreachable branch, or allow and validate the optional remaining account before rejecting unexpected remaining accounts.

Definitive Finance: Fixed in [PR 105](#).

Cantina Managed: Fix verified.

3.2.4 Out-of-range Flash output fee rates can wrap on cast instead of failing closed

Severity: Low Risk

Context: [handle_flashpostswap.rs#L84-L93](#)

Summary: Flash output fee rates are exposed as raw `u64` instruction arguments and are not capped before fee calculation. When the computed `u128` fee quotient exceeds `u64::MAX`, the final `as u64` cast truncates the value, causing invalid fee settings to continue with a wrapped effective fee instead of failing closed. The program should reject any fee rate exceeding `FEE_DIVISOR_EXTENDED`.

Finding **Description:** `programs/trading/src/utils/consts.rs:12` defines `FEE_DIVISOR_EXTENDED` as `1_000_000_000_000`. `programs/trading/src/lib.rs:115-145` exposes `output_integrator_fee_bps` and `output_protocol_fee_bps` as raw `u64` arguments. The production post-swap handler computes output fees in `programs/trading/src/handlers/handle_flashpostswap.rs:83-92` using:

```
((rate as u128) * (output as u128) / FEE_DIVISOR_EXTENDED as u128) as u64
```

The same pattern appears in event fee emission at `programs/trading/src/handlers/handle_flashpostswap.rs:183-191` and in the simulation handler at `programs/trading/src/handlers/handle_flashpostswapsim.rs:70-84`. If the exact `u128` quotient is greater than `u64::MAX`, Rust's `as u64` conversion keeps only the low 64 bits. Later settlement logic at `programs/trading/src/handlers/handle_flashpostswap.rs:148-153` then uses the wrapped value in `checked_sub`, rather than rejecting the invalid fee rate or oversized computed fee.

The root cause is missing fail-closed validation between untrusted fee-rate inputs and settlement arithmetic. The control flow widens to `u128` for multiplication, narrows with a lossy cast, and only then applies checked subtraction. Invalid fee configurations should be rejected before settlement, not converted into smaller effective fees.

Recommendation: Validate output fee rates before calculating fees. Enforce per-rate and combined policy caps, and replace lossy casts with checked conversion.

Definitive Finance: Fixed in [PR 98](#).

Cantina Managed: Fix verified.

3.2.5 Unsigned group selection can route Flash integrator fees into a performer-owned group

Severity: Low Risk

Context: [handle_flashpreswap.rs#L84-L88](#)

Description: The Flash order signature does not include the selected `group` .

This means a user can sign one Flash order, but a valid performer can execute that same signed order with a different group account. Since Flash integrator fees are routed through token accounts derived from the supplied group, the performer can choose a group they control and make the integrator fees accrue there.

Recommendation: Include the `group` identity, or the exact fee recipient accounts, in the signed Flash order message.

Definitive Finance: Fixed in [PR 95](#).

Cantina Managed: Fix verified.

3.2.6 Flash swap Ed25519 verification excludes multisig token owners

Severity: Low Risk

Context: [handle_flashpreswap.rs#L90-L95](#)

Description: `flashPreSwap()` binds off-chain approval through `verify_ed25519_instruction()` , which requires the Ed25519 pubkey in the preceding instruction to match `swapper_token_account.owner` . That owner must personally sign the constructed `DFS|m=...|t=...|h=...` message. Solana has no EIP-1271-style on-chain signature validation for program-owned accounts, so a multisig or vault PDA cannot satisfy this check even when it owns the delegated `swapper_token_account` . Integrators whose swapper is a multisig cannot use flash orders unless they route ownership through a single-key wallet that can produce the required Ed25519 signature.

Recommendation: Adopt Drift's split-authority model ([Swift docs](#)): keep the token owner as `taker_authority` (funds/delegation) and register a separate `signing_authority` that `verify_ed25519_instruction()` checks instead of `swapper_token_account.owner` . Until that exists, document in code and integrator docs that flash orders require a standard Ed25519 signer, and that multisig owners must use a dedicated trading wallet (or future delegate registration), not the multisig PDA.

Definitive Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.3 Informational

3.3.1 FlashOrders initialization can be blocked by pre-funding the PDA

Severity: Informational

Context: [handle_init_flash_nonce_tracker.rs#L19-L31](#)

Description: In `handlers/handle_init_flash_nonce_tracker.rs` , `init_flash_orders()` initializes the global `flash_orders` PDA through `system_program::create_account()` . This CPI only succeeds when the destination account is completely uninitialized, including having zero lamports. The handler writes the Anchor discriminator only after `create_account()` succeeds, so it has no fallback for a system-owned PDA that already holds lamports.

An attacker can derive the deterministic `FLASH_ORDERS_SEED` PDA and transfer a minimal amount of SOL to it before initialization. Once the PDA has lamports, `system_program::create_account()` reverts with `SystemError::AccountAlreadyInUse` . This will block `FlashOrders` initialization, which prevents the flash order nonce tracker from being created and disables the flash swap flow indefinitely.

Since the program is already upgraded and the account is already created on-chain, the attack is not possible anymore.

Recommendation: Consider replacing the current initialization logic with this [best-practice example](#) for PDA initialization.

Definitive Finance: Fixed in [PR 91](#).

Cantina Managed: Fix verified.

3.3.2 Redundant performer validation increases flash swap complexity

Severity: Informational

Context: [handle_flashpostswap.rs#L113](#), [handle_flashpreswap.rs#L138](#)

Description: In `handlers/handle_flashpreswap.rs` and `handlers/handle_flashpostswap.rs`, `process()` checks `meta_config.valid_performer(signer.key())` inside fee-routing logic. The same condition is already enforced by the `signer` account constraint in both `FlashPreSwap` and `FlashPostSwap`. Since invalid performers cannot enter the handler, the later checks always evaluate to true.

Recommendation: Remove the redundant `meta_config.valid_performer(signer.key())` checks from the handler body and rely on the existing account constraints.

Definitive Finance: Fixed in [PR 94](#).

Cantina Managed: Fix verified.

3.3.3 Deprecated library method is used during FlashOrders migration

Severity: Informational

Context: [handle_migrate_flash_orders.rs#L18](#)

Description: In `handlers/handle_migrate_flash_orders.rs`, `process()` grows the `flash_orders` PDA with `flash_orders_info.realloc(new_size, true)`. The library marks `AccountInfo::realloc()` as deprecated and recommends `AccountInfo::resize()` instead.

Recommendation: Replace `realloc()` with the supported `resize()` method while preserving the same target size and initialization behavior.

Definitive Finance: Fixed in [PR 93](#).

Cantina Managed: Fix verified.

3.3.4 Flash swap cache is left dirty after post-swap

Severity: Informational

Context: [handle_flashpostswap.rs#L13-L29](#)

Description: In `handlers/handle_flashpostswap.rs`, `process()` reads `TradingGroup.active_swap_data` to validate the cached flash output destination and balance, but it does not clear this cache after the swap completes. This leaves stale values in `TradingGroup`. Since the dirty state is unused, this is primarily a code hygiene problem.

Recommendation: Clear the flash-specific fields in `TradingGroup.active_swap_data` after successful `flash_post_swap()` settlement, or document that the fields are intentionally stale and must only be trusted within the active pre/post swap pair.

Definitive Finance: Fixed in [PR 92](#).

Cantina Managed: Fix verified.

3.3.5 Performer role is over-broad

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The program uses one broad `valid_performer()` role for many different actions.

A key that is allowed to execute Flash swaps is also accepted by other performer-gated paths, including Flash nonce closing, normal swap flows, bridge setup, and allowlisted CPI. These are different types of authority, but the program treats them as the same role.

This increases blast radius. If a performer key is compromised, stale, or only intended for Flash execution, that key can still access every handler that uses the same `valid_performer()` check. In the Flash scope, the clearest issue is that a Flash performer can also close shared Flash nonce slots, even though nonce management is a different responsibility from executing user-signed orders.

Recommendation: Split the broad performer role into narrower permissions.

Definitive Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.3.6 Flash events attribute swaps to the performer and report gross output

Severity: Informational

Context: [handle_flashpostswap.rs#L178-L196](#)

Description: `FlashSwapEvent` and `FlashSwapSimEvent` emit the executing performer as `user`, even though the authenticated source user is the owner of the delegated swapper token account. On fee-bearing settlements, the event `amount` is the gross output measured before output fees and final user receipt effects, not the final net amount received by the user.

Recommendation: Emit distinct fields for performer and authenticated source user. For example, add `performer`, `source_owner` or `swapper`, `swapper_token_account`, and `output_token_account` to the Flash events and IDL.

Definitive Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.3.7 Flash swap user signature does not bound output amount

Severity: Informational

Context: [handle_flashpreswap.rs#L76-L88](#)

Description: In `process()` for `flash_pre_swap()`, the swapper must pass an Ed25519 check over `DFS|m={mint}|t={order_total}|h={order_hash}`. The `order_hash` only commits to `output_token_account`, `nonce`, and `deadline` (plus domain `DFS_ORDER_V1`), alongside visible `input_mint` and `order_total`.

The design assumes the swapper fully trusts the performer for execution price and final output amount, and that the protocol may fulfill one signed order across multiple transactions. For those reasons, `min_output_amount` is not part of the signed payload.

Typical swap flows let users set a minimum receive so bad fills revert; this flow is input-authorization only. That is workable but unusual and should be documented as an explicit trust assumption.

Recommendation: Document for integrators and end users that flash orders authorize input debits, not guaranteed receive size, and that a single signature may cover multiple partial fills.

Definitive Finance: Acknowledged.

Cantina Managed: Acknowledged.